

Core Concepts — Why LLM Evaluation Is a Different Discipline

Part 15 —  AI Evaluation · 15-ai-evals/notes/01-core-concepts.md

Module 01 taught you evaluation as a solved problem with sharp edges: pick a metric that matches the cost of being wrong, split your data honestly, hunt leakage, distrust a bare accuracy number. All of that is still true. What changes with LLMs is that the *output space* stops being a fixed set of labels and becomes open-ended natural language, the *grader* is often itself a model with its own failure modes, and the thing you're measuring against — a benchmark — decays the moment it becomes famous enough to matter. This note is the "why" and the vocabulary; note 02 is the "how."

1. Three things that make LLM eval structurally harder

1. The output space is open-ended. A classifier emits one of k labels; you check equality. An LLM emits one of an effectively unbounded set of token sequences, many of which are correct, differently phrased, or correct-but-incomplete. `exact_match` is nearly useless above the "does it output a single token" tier — two answers can be semantically identical and string-different, or string-similar and factually opposite. This is why LLM eval reaches for **graders**: programs or models that map free text to a score, and why the grader's own reliability becomes half the problem (§3 below, and all of note 02 §3).

2. The eval is itself a model, and inherits model failure modes. The moment you use an LLM to judge an LLM's output — because a five-line assertion can't capture "is this a good explanation" — you have imported hallucination, positional bias, verbosity bias, and prompt sensitivity into your *measurement instrument*, not just your system under test. A biased ruler that's biased in the same direction as the thing it's measuring is the worst kind of error: it doesn't show up as noise, it shows up as false confidence. This is the entire subject of `code/judge_bias_and_calibration.py` and note 02 §3 — treat it as a first-class engineering problem, not a footnote.

3. Public benchmarks decay, and decay in a way you can't see from the leaderboard. A benchmark score is only informative if the model hasn't seen the answers. Every popular benchmark eventually leaks into pretraining data — scraped from a blog that quoted it, mirrored on GitHub, included in an aggregator dataset — and once that happens the benchmark stops measuring capability and starts measuring memorization. This is **contamination**, and it operates at a scale classical ML leakage never had to worry about: your train/test split leaks; a benchmark leaks into the training corpus of every future model that scrapes the public internet, including ones that don't exist yet. There is no way to "just not do that" the way you avoid leakage in Module 01 by controlling your own pipeline — you don't control the model vendor's pretraining data. The practical response is the same instinct as leakage-hunting, redirected: build your **own** eval set from your own production traces (§4 below), rotate or hold back portions of any set you publish, and treat a suspiciously high benchmark score with the same suspicion Module 01 taught you to treat a suspiciously high CV score.

Say it out loud: an eval score's validity has an expiration date. A benchmark that's two years old and famous is worth less than a boring, unglamorous 200-example set scraped from your own production failures last month.

2. Benchmark literacy: know what each one actually measures, and what it doesn't

You will be handed a benchmark score in a vendor deck or a model card. Knowing what it measures — and its specific, well-documented weaknesses — is table stakes for not being fooled by it.

Benchmark	Measures	Known weaknesses
MMLU (Massive Multitask Language Understanding)	Multiple-choice knowledge across 57 subjects	Answer-choice shortcuts (models exploit option patterns without reading the question); documented label errors in a meaningful fraction of questions; heavily saturated by frontier models, compressing differences into noise
HumanEval	Python function synthesis from a docstring	164 hand-written problems — small enough to memorize wholesale; measures <i>toy function</i> synthesis, not real-repo software engineering (no multi-file context, no existing codebase conventions)
SWE-bench	Resolving real GitHub issues in real repositories with a patch that passes hidden tests	Much closer to real engineering work than HumanEval; still narrow to a handful of popular Python repos, so it rewards familiarity with <i>those specific codebases'</i> idioms; "verified" subset exists because the original had unsolvable/ambiguous issues
GPQA (Graduate-Level Google-Proof Q&A)	PhD-level science questions written to resist search-engine lookup	Small (~450 questions), so a handful of ambiguous items move the score meaningfully; "Google-proof" was true at construction time, not necessarily after two years of models training on more science text
Chatbot Arena / LMSYS	Crowd-sourced pairwise human preference across models	Preference is not correctness — a confident, well-formatted wrong answer beats a hedgy right one; population of voters skews toward a specific demographic and prompt style; vulnerable to models tuned specifically for "sounds good" over "is good"

The general pattern: every benchmark measures a proxy, every proxy can be gamed once it's a target (Goodhart's law, same as Module 01 §on optimizing metrics you don't actually want), and every popular benchmark accumulates a specific, documented list of gaming vectors over its public lifetime. Read the benchmark's own limitations section before quoting its number in a deck. If you can't find one, that's a red flag about the benchmark, not a reason to trust the number more.

Contamination detection, briefly: the two practical techniques are (a) comparing a model's performance on a benchmark's public split vs. a freshly-constructed, unpublished set of equivalent difficulty (a real gap is diagnostic), and (b) checking whether a model can complete benchmark items *verbatim* given only a short prefix (n-gram overlap / exact continuation) — a model that has memorized the exact next line of a famous

MMLU question was trained on it. Neither is airtight; treat "this model wasn't in the training data" as a hypothesis you weaken evidence against, never a fact you confirm.

3. What "generic" benchmark scores can't tell you about your system

MMLU tells you about the base model's knowledge. It tells you nothing about whether your RAG pipeline retrieves the right chunk, whether your agent picks the right tool on turn 6, whether your customer-support bot leaks a competitor's pricing when asked sideways, or whether your summarizer drops the one clause that matters in a contract. **A generic benchmark is a proxy for "is this a capable model," and your product is not "is this a capable model" — it's a specific, narrow task wrapped around one.** The gap between those two things is exactly where custom evaluation lives, and it's the majority of the actual work in this module.

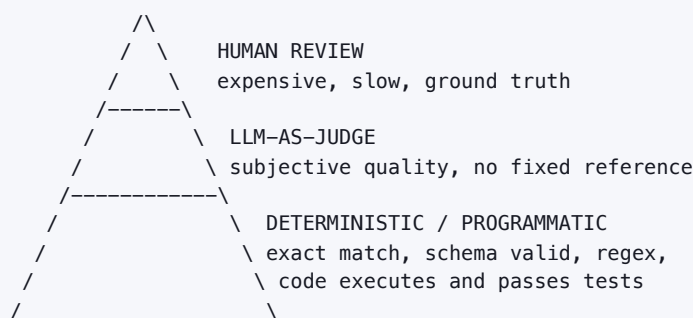
4. Building a custom eval suite: start from failures, not from imagination

The single most common mistake teams make is designing an eval suite in a conference room before they have any users. It reliably produces a suite that tests what the team *imagines* goes wrong, which is almost never what actually goes wrong. The correct order:

1. **Ship something, even a rough version, to real usage** (internal dogfooding counts if you have no users yet).
2. **Harvest real failures:** negative user feedback, error/exception traces, latency outliers, spot-checks by a human reading transcripts. This is the same "read your worst 100 predictions" habit from Module 01 §8 — LLM eval calls it a **failure taxonomy**, and it's the highest-leverage single habit in this module.
3. **Turn each distinct failure cause — not each failed example — into a rule or a rubric criterion.** Ten examples that all fail because the model doesn't cite sources are one fix, not ten data points.
4. **Only then** supplement with synthetic/adversarial cases for coverage of edge shapes you haven't hit yet in production — useful for breadth, useless as your primary signal, because synthetic data reflects what you imagined, same problem as step 0.

The eval pyramid

Not every check deserves a model call. Spend judge/human budget only where cheaper checks can't answer the question:



Climb the pyramid only when the layer below genuinely can't answer the question. "Is this JSON valid" is a schema check, full stop — routing it through an LLM judge is slower, costs money, and adds a source of noise to a question that had a deterministic answer. "Is this explanation pedagogically clear" has no programmatic check — that's where a calibrated judge earns its cost. "Did we get sued" is a human review question, always. Teams that skip straight to LLM-as-judge for problems a five-line assertion would settle end up debugging their judge for a month instead of shipping the five-line assertion.

Golden dataset sizing

"How many examples do I need?" is a statistical power question, not a vibe. The blunt version, expanded with real formulas in note 02 §4 and demonstrated in `code/statistical_rigor_for_evals.py`:

Eval set size	What it's good for	What it can't do
10-20	Smoke test — did the change break something obviously	Detect anything short of catastrophic regression
50-100	Directional signal, judge calibration against human labels	Distinguish a 2-point accuracy delta from noise
200-500	Reliable regression detection for meaningful (5+ point) deltas	Resolve small (1-2 point) deltas confidently
1,000-2,000+	Resolve small deltas; support slicing by segment without each slice going underpowered	—

A golden dataset also needs **coverage**, not just count: stratify by the failure taxonomy from step 3 above, and keep enough examples per stratum (segment, difficulty, input type) that a regression in one slice isn't averaged away by the rest. An improvement on 800 easy examples hiding a regression on 20 hard ones is a real, common way "the eval went up" ships a worse product for your highest-value users.

5. Where this returns

Idea here	Where it returns
Failure taxonomy / read your worst examples	Module 01 §8 — the same habit, this is its LLM-era name
Eval pyramid (deterministic -> judge -> human)	Note 02 §1-3 — the mechanics of each layer
Benchmark contamination	Module 04 (<code>../04-llm/</code>) — pretraining data and why leakage happens upstream of you
Golden dataset sizing	<code>code/statistical_rigor_for_evals.py</code> and note 02 §4 — the actual math
LLM-as-judge, briefly	Note 02 §3 and <code>code/judge_bias_and_calibration.py</code> — bias catalog and calibration
RAG-specific metrics (faithfulness, context precision/recall)	<code>../08-rag/</code> — RAGAS and friends; not repeated here
Agent trajectory evaluation	<code>../06-ai-agents/notes/02-planning-failure-and-evals.md</code> — step-level correctness, not repeated here
Red-team / adversarial evals	<code>../13-ai-security/</code> — the security-specific eval suite
Audit trails for eval results	<code>../12-ai-governance/</code> — what a regulator or auditor wants to see
The tooling to run all of this at scale	<code>../16-ai-tech-stack/tracks/langsmith/</code> — datasets, <code>evaluate()</code> , CI gates, online evaluators